Project 1: SIMD Advantage Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

[**TODO:** Two to four sentences: CPU and ISA studied, headline speedups per kernel, where SIMD gains compress as the workload becomes memory-bound, and the negative result.]

Contents

1 Introduction **3**

2 Methodology **3**
2.1 Platform . . . 3
2.2 Kernels . . . 3
2.3 Build variants . . . 3
2.4 Timing and statistics . . . 4
2.5 Data initialization and working-set sizes . . . 4

3 Correctness Validation **4**

4 Vectorization Verification **5**
4.1 Compiler report . . . 5
4.2 Disassembly . . . 5

5 Scalar vs. SIMD Baseline **5**
5.1 Analysis . . . 6

6 Locality Sweep **6**
6.1 Analysis . . . 7

7 Alignment and Tail Handling **7**
7.1 Analysis . . . 7

8 Stride and Gather Access **8**
8.1 Analysis . . . 8

9 Data Type Comparison **9**
9.1 Analysis . . . 9

10 Roofline Interpretation **9**
10.1 Analysis 9

11 Negative Example **11**
11.1 Evidence 11
11.2 Explanation 11

12 Anomalies and Limitations **11**
12.1 Unexpected results 11
12.2 Limitations 11

13 AI-Use Note **11**

14 Conclusion **12**

A Reproducibility **12**

Table 1: Measurement environment.

CPU	?? (??)
SIMD ISA	??
Topology	??; NUMA: ??
SMT	??
Caches	??
Memory	??; DIMMs: ??
Pages	base ??; THP: ??
OS / kernel	??; ??
Frequency policy	governor: ??; driver: ??; turbo: ??; EPP: ??
Pinning / isolation	[**TODO:** e.g. `taskset -c 2` on a P-core; isolcpus if used]; ??
Compilers	??; ??
Tools	?? (perf_event_paranoid ??); ??; MLC: ??
GPU (optional)	??; CUDA ??
Collected	??

1 Introduction

[**TODO:** Motivation: what SIMD offers on this CPU (ISA extensions, vector width, lanes per float32/float64) and why measured speedups differ from the lane count.]

This report answers:

1. How much faster are auto-vectorized builds than scalar builds for each kernel, and how does that change from cache-resident to DRAM-resident working sets? (Sections 5 and 6)
2. What do alignment, loop tails, stride and gather access, and data type cost? (Sections 7 to 9)
3. Where do these kernels sit in a roofline model of this machine? (Section 10)
4. When does the expected SIMD benefit fail to appear? (Section 11)

[**TODO:** One-paragraph summary of the main findings.]

2 Methodology

Rubric — Baseline, correctness, reproducibility (15 pts). Reference validation, controlled timing, reproducible scripts and a documented environment (with Sections 3 and 5).

2.1 Platform

[**TODO:** Vector widths actually usable on this CPU (SSE / AVX2 / AVX-512), and whether the compiler prefers 256- or 512-bit vectors by default.]

2.2 Kernels

[**TODO:** Justify the FLOP counts (and how FMA is counted). State which kernels were selected.]

2.3 Build variants

[**TODO:** Primary compiler and version; floating-point contraction setting; whether a second compiler was compared.]

Table 2: Kernels and floating-point operations (FLOPs) per processed element.

Kernel	Definition	FLOPs / element
SAXPY	$y_i \leftarrow a\,x_i + y_i$	??
Dot product	$s \leftarrow \sum_i x_i\,y_i$	??
Elementwise multiply	$z_i \leftarrow x_i\,y_i$	??
3-point stencil	$y_i \leftarrow a\,x_{i-1} + b\,x_i + c\,x_{i+1}$	??

Table 3: Build variants. Only the kernel translation unit differs; the timing harness and the reference implementation are identical across variants.

Variant	Kernel compiler flags (from `simd_bench -info`)
scalar	[**TODO:** e.g. `-O3 -fno-tree-vectorize`]
autovec	[**TODO:** e.g. `-O3`]
native	[**TODO:** e.g. `-O3 -march=native`]
native512	[**TODO:** e.g. `-O3 -march=native -mprefer-vector-width=512`, if AVX-512 is available]

2.4 Timing and statistics

- Timer: `clock_gettime(CLOCK_MONOTONIC_RAW)`; each timed sample repeats the kernel until it lasts at least ?? ms. [**TODO:** justify relative to timer resolution]
- Warm-up samples: ??; repetitions per configuration: ??.
- Reported statistic: median; error bars show the interquartile range. [**TODO:** justify]
- Pinning, SMT, governor and turbo: Table 1. [**TODO:** anything not captured there]
- Configuration order: [**TODO:** sequential or shuffled, and why]

2.5 Data initialization and working-set sizes

[**TODO:** Value distribution and seed; why no denormals, NaNs or overflow; gather index pattern.]
[**TODO:** How N maps to bytes touched (arrays $\times$ element size), and which N fall in L1, L2, LLC and DRAM on this CPU.]

3 Correctness Validation

[**TODO:** The reference implementation (plain scalar loops compiled with vectorization and FP contraction disabled) and how every variant is compared against it before timing.]

Cases	kernels $\times$ {float32, float64}; N around multiples of the lane count; start offsets; strides; gather; int32 multiply
Tolerance	[**TODO:** relative error bound and why (FMA contraction, reassociated reductions)]
Command	`make test`

[**TODO:** Known-answer sanity checks that do not depend on the reference (e.g. dot product of all-ones vectors equals N).]
Result: ?? of ?? validation cases pass for every variant and compiler.

Table 4: SIMD instructions per kernel function and build variant.

Data pending: `data/vectorization.csv`
run the experiment in ACS-Code, then `scripts/sync_data.sh`

4 Vectorization Verification

Rubric — Vectorization verification (15 pts). Clear evidence that SIMD instructions are used, with a correct reading of vector width, FMA and masking.

Hypothesis (written before measuring)

[**TODO:** Which loops you expect each variant to vectorize, at what width, and which you expect the compiler to reject (and why).]

Table 4 counts, for each kernel function, the instructions that operate on vector registers or use packed operations (`make report`).

4.1 Compiler report

```
TODO: 2-5 relevant lines from data/raw/vectorization/<run>/report_<variant>.txt
```

4.2 Disassembly

```
TODO: 5-10 lines of a vectorized loop body, e.g. disasm/native/saxpy_f32.s
```

[**TODO:** Interpret the evidence: register width (xmm/ymm/zmm) and lanes per element type; FMA (`vfmadd*`); masked tails (k-mask registers) versus a scalar epilogue; loops that did not vectorize and the compiler's reason.]

5 Scalar vs. SIMD Baseline

Rubric — Baseline, correctness, reproducibility (15 pts). Speedup and achieved throughput for every kernel across working-set sizes.

Hypothesis (written before measuring)

[**TODO:** Expected speedup per kernel relative to the lane count, and how it should change between cache-resident and DRAM-resident N.]

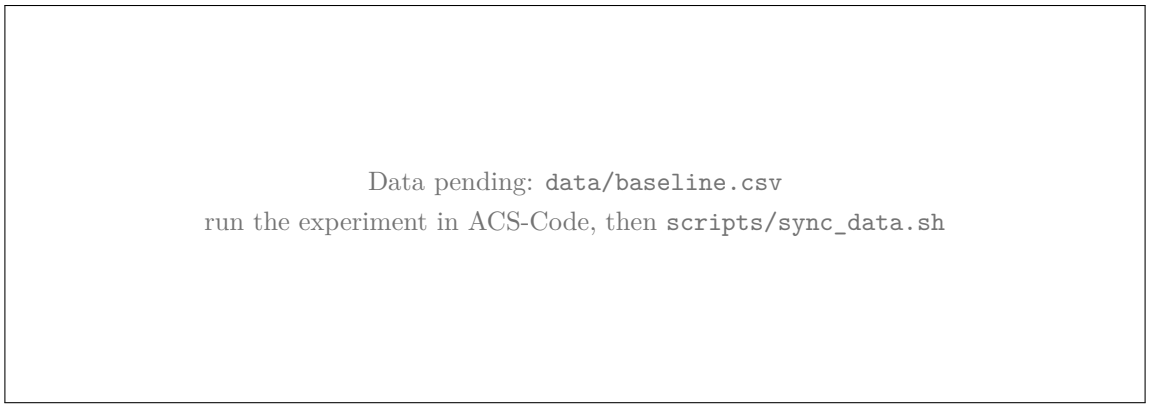


Figure 1: Speedup of the SIMD builds over the scalar build. [**TODO:** one-sentence takeaway]

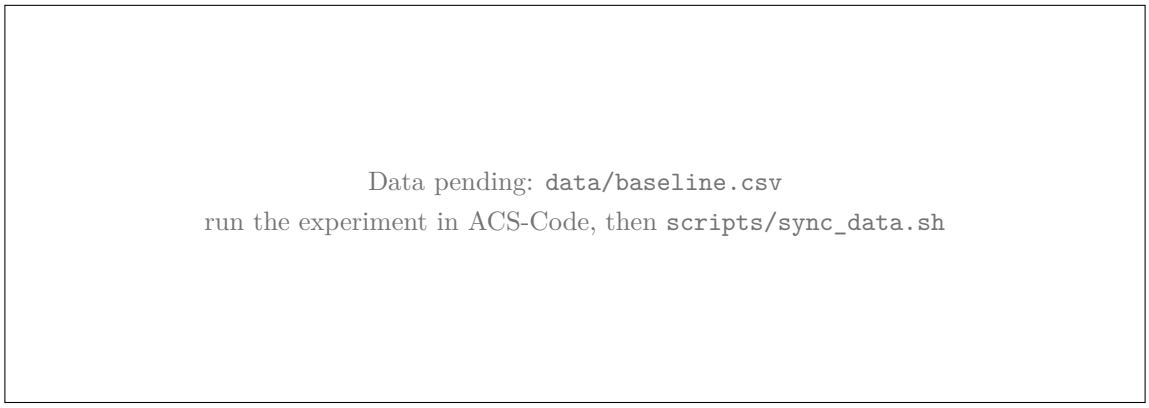


Figure 2: Achieved throughput. [**TODO:** one-sentence takeaway]

Fixed	float32; unit stride; aligned arrays; one thread pinned to CPU ??
Varied	kernel; variant (scalar, autovec, native); $N = 2^8 \dots 2^{26}$
Repetitions	?? per point; median, IQR error bars
Script	`SIMD_Advantage_Profiling/scripts/run.sh baseline`

5.1 Analysis

[**TODO:** Speedups versus lane count per kernel; why reductions (dot) and stencils behave differently from SAXPY and elementwise multiply; where the curves bend and why.]

6 Locality Sweep

Rubric — Locality and memory-bound analysis (20 pts). Working-set transitions identified, and the compression of SIMD gains in the memory-bound regime explained.

Hypothesis (written before measuring)

[**TODO:** At which N each cache transition should appear for this kernel, and how scalar and SIMD builds should diverge or converge beyond LLC.]

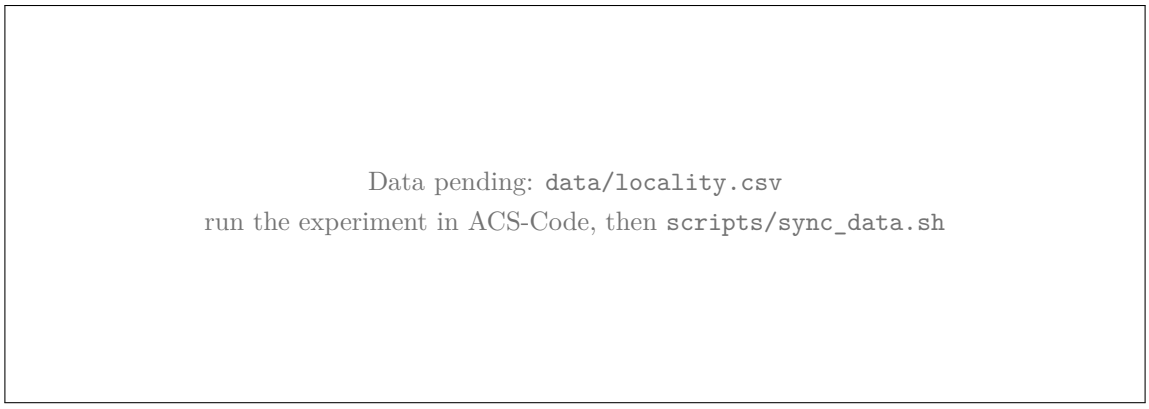


Figure 3: Time per element across cache levels. [**TODO:** takeaway]

Fixed	kernel ??; float32; unit stride; aligned; pinned to CPU ??
Varied	N (dense grid across L1/L2/LLC/DRAM); variant
Counters	[**TODO:** perf events recorded (`PERF=1`), e.g. LLC-load-misses]
Script	`SIMD_Advantage_Profiling/scripts/run.sh locality`

6.1 Analysis

[**TODO:** Where each transition occurs (bytes = arrays × element size ×N) and what the counters show there; why the SIMD speedup compresses once memory traffic dominates.]

7 Alignment and Tail Handling

Rubric — Alignment, tail, stride/gather studies (20 pts). Quantified alignment, prologue/epilogue and masking effects with a convincing explanation (with Section 8).

Hypothesis (written before measuring)

[**TODO:** Expected cost of misaligned starts on this CPU, and how time per element should vary when N is not a multiple of lanes × unroll factor.]

Fixed	kernel ??; float32; unit stride
Varied	start offset from a 64-byte boundary (0–32 bytes); N just above multiples of the vector width
Script	`SIMD_Advantage_Profiling/scripts/run.sh alignment`

7.1 Analysis

[**TODO:** Size of the prologue/epilogue cost relative to the loop body; aligned vs. unaligned loads on this microarchitecture; masked tails (AVX-512) vs. scalar remainder loops, backed by disassembly.]

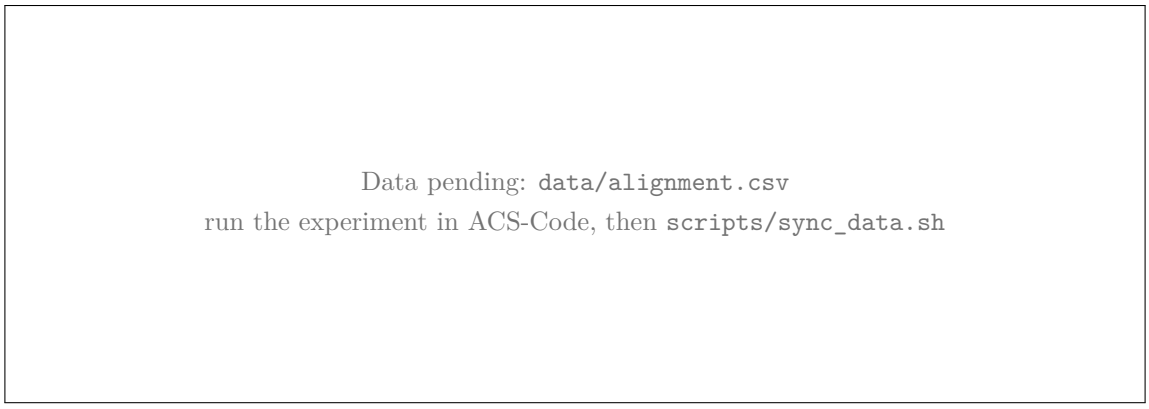


Figure 4: Alignment and tail-handling costs. [**TODO:** takeaway]

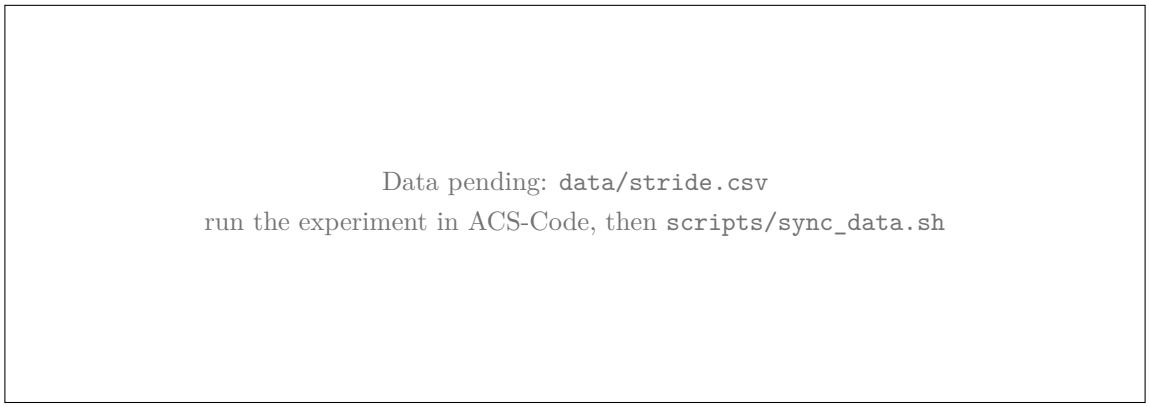


Figure 5: Effect of stride on time per processed element. [**TODO:** takeaway]

8 Stride and Gather Access

Hypothesis (written before measuring)

[**TODO:** How strides of 2/4/8 and gather-style indexing should change time per processed element and SIMD efficiency, in cache and in DRAM.]

Fixed	SAXPY; float32
Varied	access (unit, stride 1/2/4/8, gather); N cache-resident and DRAM-resident; variant
Definition	[**TODO:** stride semantics used (e.g. $i = 0, s, 2s, \ldots < N$) and gather index pattern]
Script	`SIMD_Advantage_Profiling/scripts/run.sh stride`

8.1 Analysis

[**TODO:** Effective memory bandwidth per processed element; whether strided loops vectorize and how (gather/scatter vs. scalar); why gather rarely pays off here.]

Data pending: `data/dtype.csv`
run the experiment in ACS-Code, then `scripts/sync_data.sh`

Figure 6: Speedup by data type. [**TODO:** takeaway]

9 Data Type Comparison

Hypothesis (written before measuring)

[**TODO:** Expected float32 vs. float64 (and int32) speedups from lane count, operation cost and bytes per element, in cache and in DRAM.]

Fixed	unit stride; aligned
Varied	data type; kernel; variant; N
Script	`SIMD_Advantage_Profiling/scripts/run.sh dtype`

9.1 Analysis

[**TODO:** Explain with lane count, arithmetic intensity (FLOPs per byte) and per-operation cost; note where float64 halves the lanes but not necessarily the speedup.]

10 Roofline Interpretation

Rubric — Roofline and quantitative insight (20 pts). Correct arithmetic intensity, use of measured memory bandwidth, and a justified compute- vs. memory-bound conclusion.

Hypothesis (written before measuring)

[**TODO:** Which kernels you expect to be compute-bound and which memory-bound, before computing the model.]

Attainable performance at arithmetic intensity I (FLOPs per byte of memory traffic) is bounded by

$$P(I) = \min(P_{\text{peak}},\ B \cdot I), \tag{1}$$

where P_{peak} is the CPU's peak FLOP rate and B the memory bandwidth [1].

10.1 Analysis

[**TODO:** Place each kernel on the model; compute- vs. memory-bound conclusion; consistency with the locality sweep (Section 6); what would move a kernel toward the roof.]

Table 5: Roofline inputs.

Quantity	Value	Source
Peak FLOP rate P_{peak}	?? GFLOP/s	[**TODO:** how estimated (e.g. cores/vector width/measured compute-bound kernel)]
Memory bandwidth B	?? GiB/s	[**TODO:** measurement: Project 2 `mem_bandwidth` or Intel MLC]
Intensity I, SAXPY	?? FLOP/byte	[**TODO:** derivation]
Intensity I, dot product	?? FLOP/byte	

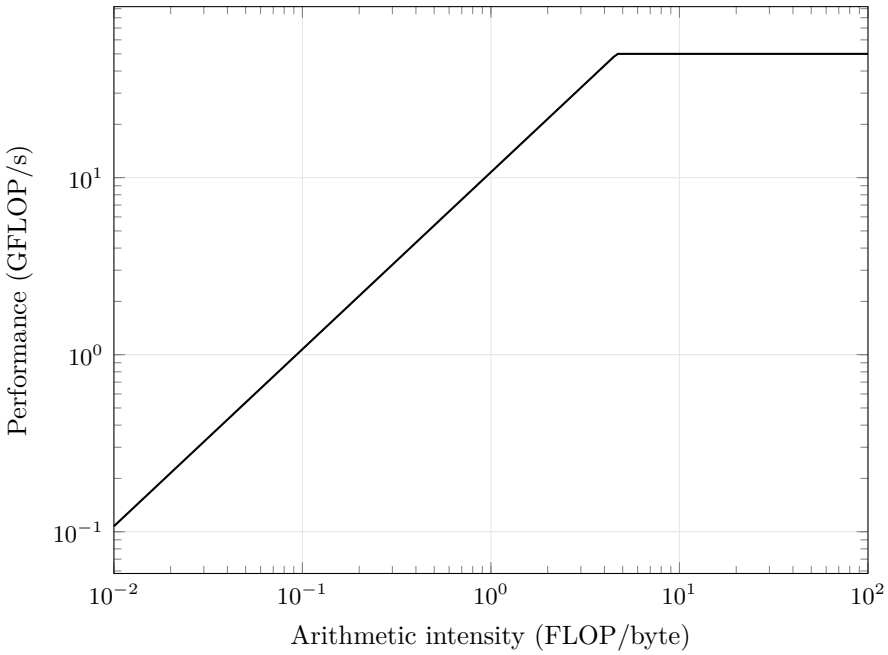


Figure 7: Roofline for this machine with measured kernels. [**TODO:** placeholder constants until measured]

11 Negative Example

Rubric — AI-use transparency and failure case (10 pts). A meaningful negative or misleading example, verified and explained (with Section 13).

Hypothesis (written before measuring)

[**TODO:** What a naive reading (or a quickly generated benchmark) would predict for this case.]

Case	[**TODO:** a loop that looks SIMD-friendly but is not vectorized, or one that vectorizes but gains little because memory bandwidth dominates]
Build	[**TODO:** variant / extra flags]
Script	`SIMD_Advantage_Profiling/scripts/run.sh negative`

11.1 Evidence

[**TODO:** Vectorization report or disassembly excerpt plus timing figure/table.]

11.2 Explanation

[**TODO:** Why the expected speedup does not appear, and how you verified the explanation.]

12 Anomalies and Limitations

Rubric — Report quality (10 pts). Clear plots, concise writing, honest anomalies and thoughtful limitations.

12.1 Unexpected results

- [**TODO:** Observation → plausible hypothesis → how it was (or could be) tested.]

12.2 Limitations

- [**TODO:** e.g. one machine and compiler version, timer and call overhead at small N, turbo and thermal state, perf counters covering the whole process.]

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

[**TODO:** Tools and models, and for which parts of the project.]

What they helped generate

[**TODO:** e.g. repository scaffolding, build and run scripts, debugging help.]

What was independently modified, tested, or rejected

[**TODO:** Concrete examples, including at least one AI suggestion that was wrong or misleading and how it was caught.]

Validation

[**TODO:** How correctness and measurements were checked independently (reference comparison, vectorization evidence, sanity checks).]

14 Conclusion

[**TODO:** Three to five sentences: when SIMD helps on this machine, when gains compress or vanish, and the single most important lesson for writing and benchmarking vectorized code.]

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785

A Reproducibility

Code, scripts and raw data: `github.com/SKempiak/ACS-Code`, commit [**TODO:** hash].

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
make && make test
scripts/collect_env.sh
SIMD_Advantage_Profiling/scripts/run.sh all
make -C SIMD_Advantage_Profiling report
scripts/sync_data.sh SIMD_Advantage_Profiling
```

Table 6: Raw data behind each figure.

Figure / table	Raw run directory (`SIMD_Advantage_Profiling/data/raw/...`)
Figures 1 and 2	[**TODO:** `baseline/<timestamp>`]
Figure 3	[**TODO:** `locality/<timestamp>`]
Figure 4	[**TODO:** `alignment/<timestamp>`]
Figure 5	[**TODO:** `stride/<timestamp>`]
Figure 6	[**TODO:** `dtype/<timestamp>`]
Table 4	[**TODO:** `vectorization/<timestamp>`]